LAB EXPERIMENT 6

Experiment 6: Bigram N-Gram Model for Text Generation

Aim

To implement a simple bigram language model that generates text using pairs of consecutive words.

Algorithm

1. Start the program.
2. Import the random module.
3. Define a small training text and split it into words.
4. Build a bigram dictionary in which each word is associated with the word or words that follow it.
5. Choose a starting word.
6. Randomly select a following word from the bigram dictionary for a fixed number of steps, stopping if no continuation exists.
7. Display the generated text and stop the program.

Code

```
import random
import re

text = "I love natural language processing. I love Python programming."
words = re.findall(r"\b\w+\b", text.lower())

bigrams = {}
for i in range(len(words) - 1):
    current_word = words[i]
    next_word = words[i + 1]
    bigrams.setdefault(current_word, []).append(next_word)

word = "i"
generated = [word]

for _ in range(8):
    choices = bigrams.get(word)
    if not choices:
        break
    word = random.choice(choices)
    generated.append(word)

print("Generated Text:", " ".join(generated))
```

Output / Screenshot

```python
import random

text = "I love natural language processing. I love Python programming."

words = text.lower().replace(".", "").split()
bigrams = {}

for i in range(len(words) - 1):
    first = words[i]
    second = words[i + 1]
    bigrams.setdefault(first, []).append(second)

word = "i"
result = [word]

for i in range(8):
    if word not in bigrams:
        break
    word = random.choice(bigrams[word])
    result.append(word)

print("Generated Text:", " ".join(result))
```

```
Generated Text: i love python programming
```

Result

Thus, a bigram model was created from the sample text and used to generate a short sequence of words.